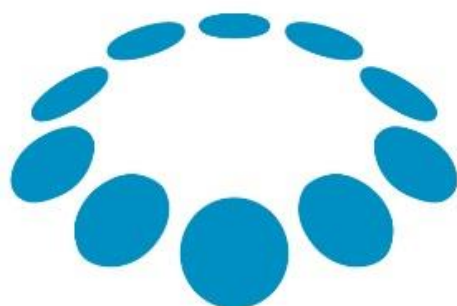




המסלול האקדמי

המכללה למינהל

להצלחה יש דרך

פתוח תוכנה מתקדם 1

תרגילי תכנות לתרגול עצמי

הקדמה

במערכת הבדיקות: <http://ck.cs.colman.ac.il> קיים הקורס PTM1.

קורס
שם המטלה
הורד מטלה
ת.ז (9 ספרות):
קבצים:
שלח

כדי להוריד את הקבצים הרלוונטיים למטלה תוכלו לבחור בה (ex1, ex2...), להקיש את ת.ז שלכם, לסמן "הורד מטלה" וללחוץ שלח. לאחר מכן תוכלו לבחור את הקבצים הנדרשים להגשה. בכל הגשה יש לבחור בכל הקבצים הנדרשים בבת אחת.

תוכלו לשלוח את הקבצים לבדיקה במוד אימון או במוד הגשה. מוד אימון מריץ את mainTrain.java שקיבלתם יחד עם המטלה – main חשוף שבודק חלק מהבדיקות. לעומתו מוד הגשה מריץ main דומה עם יותר בדיקות והוא חסוי.

במוד אימון תקבלו משוב מלא של שגיאות קומפילציה, ריצה ושגיאות לוגיות. לעומת זאת במוד הגשה רק תקבלו הודעה כל כך שקיימות שגיאות קומפילציה או ריצה. במידה ואין כאלה הבדיקה תמשיך ותקבלו פירוט של שגיאות לוגיות במידה וישנן.

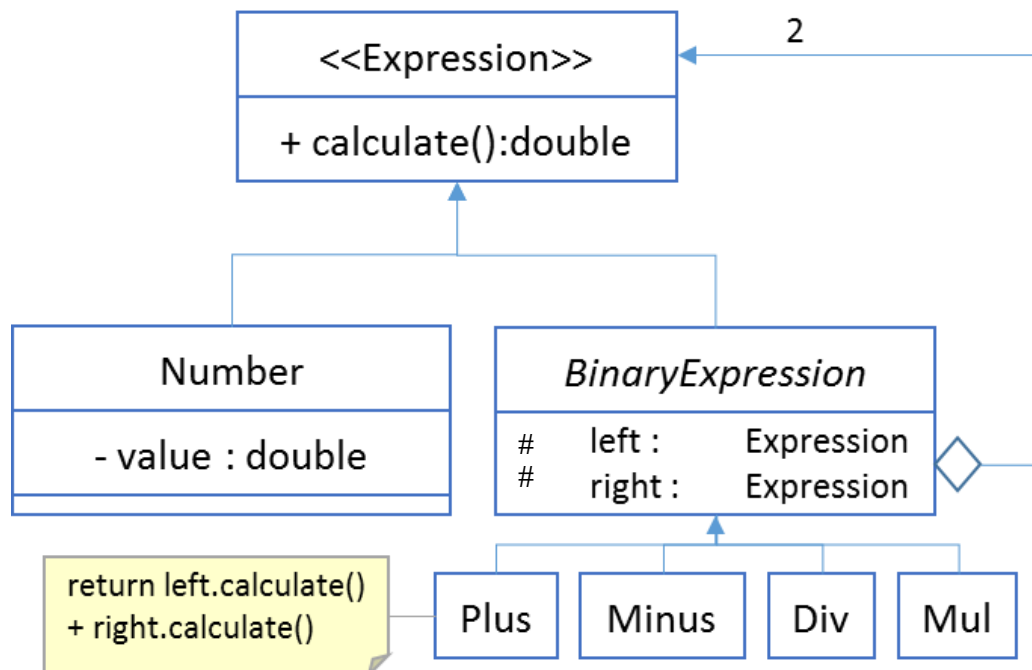
בקורס זה אין חובת הגשה לתרגילים הבאים, הם אינם מהווים חלק מהציון לקורס, והם מיועדים אך ורק לתרגול עצמי ולפיכך נדרשת משמעת עצמית שלכם. הפתרונות לתרגילים אלה יפורסמו בבוא העת.

בהצלחה!

אלי.

תרגיל 1 – class diagram

נתון תרשים ה class Diagram הבא, ממשו את הטיפוסים השונים המוצגים בו במחלקות
Java ב package בשם test.



הערה: יש לממש את המחלקות השונות ולא רק ליצור אותן ריקות.

הגשה ל: ex1

קבצים נדרשים:

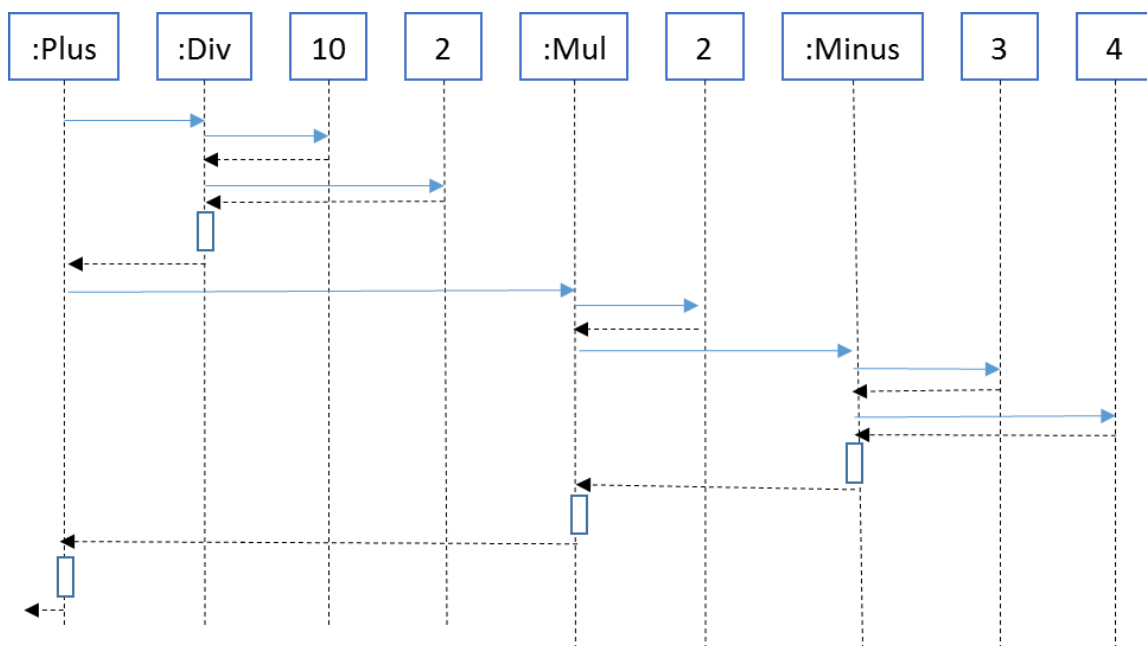
Expression.java, Number.java, BinaryExpression.java, Plus.java, Minus.java,
Div.java, Mul.java

תרגיל 2 – sequence diagram

ב UML ישנו סוג של תרשים שנקרא sequence diagram המבטא רצף כלשהו של פעולות. המלבנים למעלה מציינים אובייקטים. לעתים קרובות לא מציינים את שם האובייקט אלא רק את הטיפוס שלו (מסומן לאחר ":"), למשל בריבוע הראשון Plus: מצוין אובייקט מסוג המחלקה Plus. הקו המקווקו האנכי שיוצא מהאובייקט מצוין את אורך חייו. חץ מאוזן רציף מצוין הפעלה של מתודה באובייקט שקיבל את החץ. שם המתודה יופיע בד"כ על החץ. בתרשים למטה למחלקות האלה יש רק מתודה אחת (calculate) ולכן זה מיותר לרשום זאת. חזרה (return) מצוינת ע"י חץ מקווקו בכיוון ההפוך, ומלבן לבן מצוין שנעשה עיבוד כלשהו.

ב package בשם test, ממשו את המתודה במחלקה הבאה כך שתריץ את מה שמגדיר ה sequence diagram הבא. כמובן יש להשתמש במחלקות שיצרתם בתרגיל 1.

```
public class Q2 {
    public static double calc(){
        return 0;
    }
}
```



הגשה ל: ex2

קבצים נדרשים:

Expression.java, Number.java, BinaryExpression.java, Plus.java, Minus.java,
Div.java, Mul.java, Q2.java

תרגיל 3 – activity diagram

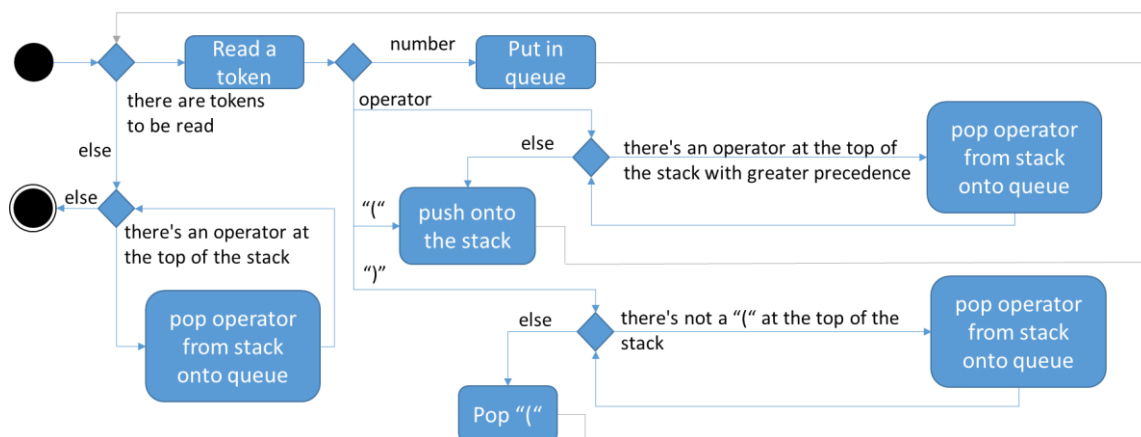
דרך נוספת לבטא רצף של פעולות ב UML הוא ע"י activity diagram. בעצם מדובר באוטומט פשוט. העיגול השחור מהווה את נקודת ההתחלה, ואילו העיגול המוקף במעגל מצין את נקודת הסיום. כל ריבוע מעוגל מציין פעילות (activity) שיש לבצע ואילו המעוין מבטא תנאי.

בהינתן מחרוזת של ביטוי, לדוג' $3+(4/2)*5$, יש לפרש אותה ולחשב את התוצאה. עבור החישוב תצטרכו ליצור את האובייקטים המתאימים, לדוג':

```
Expression e=new Plus(new Number(3) , new Mul( new Div(new Number(4), new Number(2)) , new Number(5)));
```

```
return e.calculate();
```

כדי שתוכלו ליצור את האובייקטים המתאימים למחרוזת עליכם תחילה לפרש ולסדר אותה. לשם כך ישנו אלג' של דייקסטרה בשם Shunting-yard, המובא לפניכם כ activity diagram.



בהינתן ביטוי infix, האלגוריתם מסדר את המספרים בתור, ומשתמש במחסנית כדי להכניס את האופרטורים לתור זה בסדר שמציג את הביטוי כ postfix. למשל עבור הדוגמא לעיל בסוף האלג' התור יראה כך: $342/5*+.$ כשנקרא את התור הפוך (כלומר, מימין לשמאל) נבין שעלינו לבצע חיבור של (הכפלה של 5 עם (חלוקה של 4 ב 2) עם 3.

לפיכך, נוכל לייצר בהתאם לביטוי את המופעים של Plus, Minus, Mul, Div, Number ולחשב את תוצאת הביטוי.

ב package בשם test, ממשו את המתודה במחלקה הבאה כך שבהינתן ביטוי כמחרוזת, תחזירו את תוצאת החישוב של הביטוי. תוכלו כמובן להשתמש במחלקות של תרגיל 1 ולהוסיף מתודות עזר למחלקה זו.

```
public class Q3 {
    public static double calc(String expression){
        return 0;
    }
}
```

הגשה ל ex3

יש להגיש את הקבצים הבאים:

Expression.java, Number.java, BinaryExpression.java, Plus.java, Minus.java,
Div.java, Mul.java, Q3.java

תרגיל 4 – Factory Pattern גנרי, ביטויי למדה

שאלה 1:

הכרנו את ה factory pattern הקלאסי שמגדיר שלכל מוצר יהיה יוצר משלו, כאשר כל המוצרים הם סוג של אותו ה"מוצר" וכל היוצרים הם סוג של אותו ה"יוצר". אחד הטריקים התכנותיים הוא להשתמש ב hashMap כדי למפות מפתחות לאותם "יוצרים". כך בהינתן מפתח נוכל לקבל ב $O(1)$ את היוצר (מבלי לדעת איזה יוצר ספציפי הוא) ולבקש ממנו ליצור את המוצר. נקבל מיד את המוצר החדש (שוב, מבלי לדעת איזה במוצר ספציפי מדובר).

לדוגמא, אם היינו מגדירים את הממשק Creator עם המתודה Employee create(); אז כל מימוש שונה של Creator היה מחזיר סוג שונה של Employee. לו היתה לנו המפה HashMap<String, Creator> creators; אז יכולנו להזין לה מפתח מסוג String ולקבל מיד את היוצר שיחזיר לנו את ה Employee הספציפי החדש שרצינו לקבל. לדוגמא:

```
creators.put("administrator", new AdminCreator());
```

...

```
Employee e = creators.get("administrator").create();
```

הפעולה creators.get("administrator") החזירה לנו Creator ספציפי שהיה שמור תחת המפתח "administrator" במפה (אובייקט מסוג AdminCreator), וממנו הפעלנו את המתודה create שהחזירה בתורה Employee חדש מסוג Administrator.

התיאור לעיל מהווה מנגנון Factory Pattern ספציפי עבור Employee. בתרגיל זה נרצה לממש מנגנון גנרי, כך שבמקום Employee, נוכל להזין כל טיפוס.

אם כך, ברצוננו ליצור את המחלקה `GenericFactory<Product>` המממשת `Factory Pattern` כללי ע"פ ההגדרות הבאות:

1. המחלקה תעשה שימוש (א) בממשק הפרטי `Creator`. שימו לב שבהגדרתו הוא אינו זורק חריגות. (ב) במפה.
 2. המתודה `insertProduct` תקבל (א) מחרוזת המהווה מפתח להשגת המוצר, ו (ב) משתנה `c` מסוג `Class` המתאים לכל סוג של `Product`. בהתאם לתבנית העיצוב, **מתודה זו תשייך לכל מוצר `c` יוצר משלו**, ותמפה את היוצר הזה למפתח.
 3. בהינתן מחרוזת מפתח, המתודה `getNewProduct` תחזיר לי אובייקט חדש מהסוג של `Product`. חזרה זו צריכה להיות ב $O(1)$. במידה והמפתח לא קיים המתודה תחזיר `.null`.
- השלימו את הקוד הבא כדי לעמוד בדרישות אלו (המקומות החסרים + 2 המתודות האחרונות).

```
package test;
```

```
public class GenericFactory<Product> {  
    private interface Creator<_____>{  
        public _____ create(); // no unhandled exceptions  
    }  
  
    Map<String,_____> map;  
  
    public GenericFactory(){  
        map=new _____<>();  
    }  
  
    public void insertProduct(String key, Class<_____> c) {  
    }  
  
    public Product getNewProduct(String key){  
        return null;  
    }  
}
```

דוגמא לשימוש במנגנון זה (המנהל, המפתח והארכיטקט הם סוגים של עובד):

```
GenericFactory<Employee> factory=new GenericFactory<Employee>();  
factory.insertProduct("manager", Manager.class);  
factory.insertProduct("developer", Developer.class);  
factory.insertProduct("architect", Architect.class);  
//...  
Employee e=factory.getNewProduct("manager");
```

שאלה 2: (ביטויי למדה)

נתונה המחלקה הבאה:

```
package test;
public class Level {

    char[][] data;

    public Level(char[][] data) {
        this.data=data;
    }

    public void changeData(int row,int col,char c){
        data[row][col]=c;
    }
}
```

עליכם לערוך אותה כך שמתודת ה main הבאה תעבוד. ברגע שמוכל שינוי ל Level השליטה צריכה לעבור לפונקציה המוזרקת. בדוגמא למטה, השליטה תעבור לפונקציית ההדפסה.

```
public static void main(String[] args) {
    char[][] data=new char[10][10];
    Level l=new Level(data);

    l.onChange=()->System.out.println("a change!");

    l.changeData(0, 0, 'a'); // output = "a change!"
}
```

הגשה ל ex4, את הקובץ GenericFacotry.java ואת Level.java

תרגיל 5 – Decorator, Adapter patterns

שאלה 1: בפרויקט אחד נתונים האלמנטים הבאים:

```
public interface Moveable {
    public enum Direction{up,down,left,right}
    boolean moveMe(Direction d);
}

public class Box implements Moveable{
    @Override
    public boolean moveMe(Direction d) {
        // TODO Auto-generated method stub
        return false;
    }
}
```

ובפרויקט אחר יש צורך באובייקטים מסוג הממשק Pushable, המגדיר פעולת תזוזה בכיוון 0..359 מעלות.

```
public interface Pushable {
    void push(int heading); // 0..359 Degrees
}
```

ב package בשם test, כתבו מחלקה בשם PushableAdapter שתממש object adapter המאפשר להמיר אובייקט כמו Box ל Pushable. הקפידו שהזנת "ה object" תהיה בבנאי של PhshableAdapter ושאינ בנאים אחרים. אופן ההמרה יהיה כך:

- מעלות 45 עד 134 יוגדר כתזוזה ימינה
- מעלות 135 עד 224 יוגדר כתזוזה למטה
- מעלות 225 עד 314 יוגדר כתזוזה שמאלה
- מעלות 315 (דרך 0) עד ל 44 יוגדר כתזוזה למעלה

יש להקפיד על עקרונות Solid.

שאלה 2: בקובץ Q2.java נתונים הממשקים StringReader ו IntArrayReader. עליכם להשלים את המחלקות הבאות:

- המחלקה MyWordFilter שבהתאם לתבנית Decorator, עליה להיות StringReader, כך שתהפוך כל קריאה של מחרוזת למחרוזת בה כל מילה מופיעה בדיוק פעם אחת, לפי סדר הופעתה, ומופרדת ע"י ";". ניתן להניח כי המילים מופרדות ע"י רווח אחד.
 - לדוגמא הביטוי במתודה testAPI שנמצאת בסוף Q2. ניתן לראות כיצד עבור "Hello Hello World World Q2" נקבל בקריאה "Hello;World;Q2;"
- המחלקה MyIntArrayToStringReader שבהתאם לתבנית Object Adapter תבצע אדפטציה מ IntArrayReader ל StringReader, כך שבהינתן מערך של int-ים היא תחזיר מחרוזת בה כל int מהווה מילה מופרדת ע"י רווח.
 - ב testAPI ניתן לראות שעבור המערך 100,100,50,50,13,14 באמצעות ה object adapter וה decorator לעיל נקבל פלט של "100;50;13;14;"
 - הסבר: ה decorator הפך את המערך למחרוזת של מילים מופרדות ע"י רווח, וה decorator החזיר מחרוזת בה כל מילה מופיעה פעם אחת, לפי סדר הופעתה, מופרדת ע"י ";".

הגשה ל ex5, את Q2.java, PushableAdapter.java

תרגיל 6 – streaming

נתונה המחלקה Level. מחלקה זו מייצגת שלב במשחק כלשהו עם דמות. מבנה השלב מיוצג ע"י מערך דו ממדי של char, ומיקום הדמות מיוצג ע"י המשתנים cx,cy.

```
package test;
```

```
public class Level {  
  
    private char[][] data;  
    private int cx,cy;  
  
}
```

א. ממשו את קוד הלקוח הבא, כך שיתחבר לשרת ב ip וב port, ישלח לו אובייקט מסוג Level **בפורמט XML** ויתנתק. שימוש לב, אין צורך ליצור קובץ XML אלא פשוט ליצור מידע בפורמט XML ולשלוח אותו לשרת. מותר לערוך שינויים במחלקה Level ע"פ הצורך.

```
public class MyClient {  
    public static void sendLevel(String ip,int port,Level level) throws  
    IOException{  
        // write your answer here  
    }  
}
```

ב. ממשו במחלקה הבאה קוד ששומר לקובץ בינארי אובייקט מסוג Level. שם הקובץ יינתן בפרמטר filename.

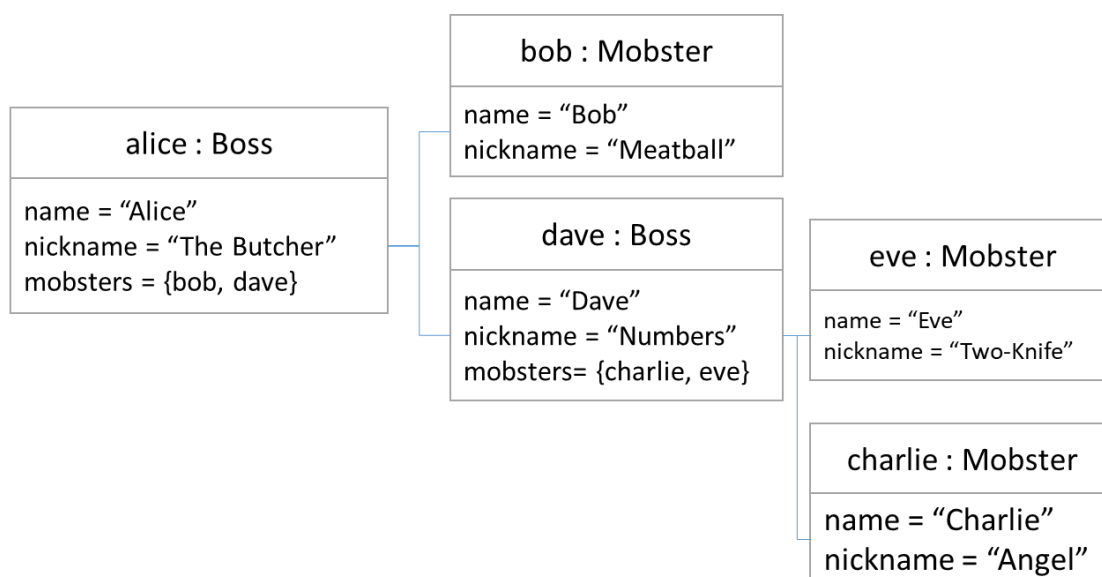
```
public class MyFiler {  
    public static void writeLevel(String fileName,Level level) throws  
    IOException{  
        // write your answer here  
    }  
}
```

הגשה ל ex6, את הקבצים Level.java, MyClient.java, MyFiler.java

תרגיל 7 – UML

שאלה 1:

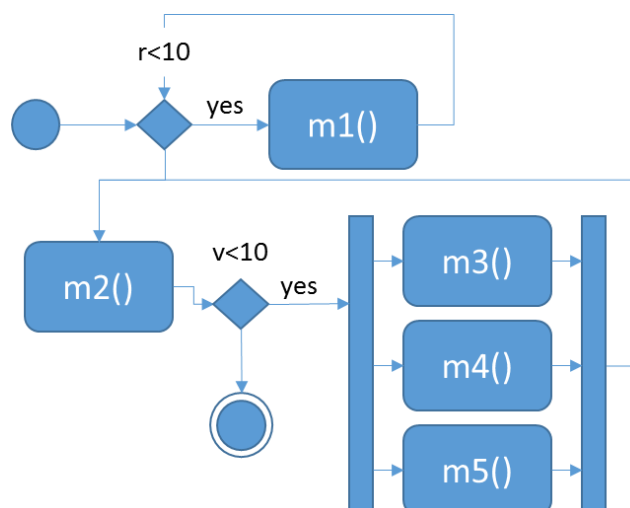
הגדירו את **שתי המחלקות** הדרושות לכך כדי שיהיה ניתן לייצר אובייקטים כמו אלה המתוארים ב Object Diagram הבא. הקפידו על שמות המחלקות והשדות בהתאם לתרשים; כאמור, הקפידו על שתי מחלקות בלבד. הגדירו אותן כמחלקות פומביות בתוך המחלקה Q1. הגדרת המחלקה כוללת את חתימת המחלקה (שם המחלקה, מה ירשה וכו') והשדות (data members) שלה בלבד, בהרשאה פומבית. אין צורך לממש מתודות כלשהן.



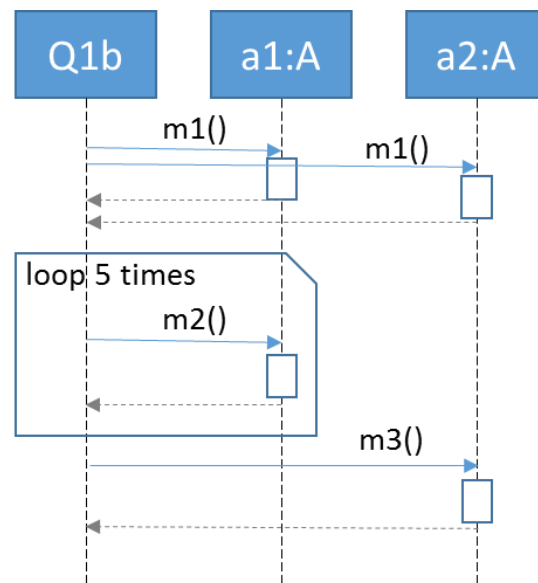
שאלה 2:

נתונה לכם המחלקה A עם המתודות `m1()` עד `m5()`. במחלקה Q2 השלימו את המתודות `a()` עבור סעיף א, ואת `b()` עבור סעיף ב'. עליכם להפעיל את המתודות של A ע"פ תרשימי ה UML הבאים:

סעיף א':



סעיף ב:



תרגיל 8 Object Adapter

בקובץ Q1.java תמצאו מבנה נתונים חדש בשם Ring המהווה רשימה מעגלית. Ring ירשה את LinkedList. אך כדי ש Ring תתנהג כרשימה מעגלית עליה לשנות את התנהגות ה iterator המקורי. לשם כך נמצאת המחלקה RingIterator בתוך המחלקה Ring. תהווה Object Adapter ע"י כך שבאמצעות הכלה של ה iterator המקורי היא תממש את הממשק הנדרש של iterator. כמובן, ל RingIterator תמיד יש next. כאשר הגענו לסוף הרשימה המקושרת, הפעולה next תוביל אותנו חזרה לאיבר הראשון. לדוגמא, ב APItest תוכלו לראות הכנסה של a,b,c,d אך מכיוון שרצנו על הרשימה עד ל $size+2$ הפלט כולל חזרה על שני האיברים הראשונים: a,b,c,d,a,b. טיפ: ירשתם גם מתודה בשם listIterator שמחזירה iterator חדש שעובר על כל האיברים כרשימה רגילה (לא מעגלית). במידת הצורך תוכלו להשתמש בה.

● **ListIterator<E> java.util.AbstractList.listIterator()**

listIterator

public [ListIterator<E>](#) listIterator()

Returns a list iterator over the elements in this list (in proper sequence). This implementation returns `listIterator(0)`.

Specified by:

[listIterator](#) in interface [List<E>](#)

תרגיל 9 – Creational patterns

שאלה 1:

ברצוננו ליצור את הטיפוס Goblin כאשר כל אובייקט מטיפוס זה הוא immutable. לכן, כל שדותיו מוגדרים כ final. אולם, רק השם והצבע הם שדות חובה. השלימו את הקוד (מקמות חסרים + מתודות ריקות) ע"פ ה Builder pattern עבור אובייקטים שהם immutable.

```
package test;

public class Goblin {

    private final String name; // required
    private final String color; // required
    private final int size; // not required
    private final byte iq; // not required

    _____ Goblin(_____ builder) {
    }

    _____ _____ class GoblinBuilder{
        private final String name;
        private final String color;

        _____
        _____

        _____ GoblinBuilder(_____ ) {
        }

        _____ _____ withSize(int size){}

        _____ _____ withIq(byte iq){}

        _____ _____ build(){
            return _____;
        }
    }

    public String getName() { return name; }

    public String getColor() { return color;}

    public int getSize() { return size;}

    public byte getIq() { return iq;}

}
```

שימוש לדוגמא:

```
//new Goblin(...); // cannot create goblins without the builder
new Goblin.GoblinBuilder("zoot", "green").withSize(500).withIq((byte)40).build();
Goblin g1=new Goblin.GoblinBuilder("danganesh", "green").withSize(600).build();
Goblin g2=new Goblin.GoblinBuilder("gilgal", "red").withIq((byte)100).build();
```

שאלה 2:

עבור הטיפוסים מהסוג של A ו B, השלימו את הקוד הבא כך ש AbstractFactory1 יחזיר מצורים מסוג "1" ואילו AbstractFactory2 יחזיר מוצרים מסוג "2".

```
public class Q2 {  
  
    public abstract class A{ public abstract String get(); }  
    public class A1 extends A{ public String get(){return "a1";} }  
    public class A2 extends A{ public String get(){return "a2";} }  
  
    public abstract class B{ public abstract String get(); }  
    public class B1 extends B{ public String get(){return "b1";} }  
    public class B2 extends B{ public String get(){return "b2";} }  
  
    public interface AbstractFactory{  
        A createA();  
        B createB();  
    }  
    // מכאן ואילך עליכם להשלים את הקוד  
    public class AbstractFactory1{ }  
  
    public class AbstractFactory2{ }  
  
}
```

הגשה ל ex9 את Goblin.java, Q2.java

תרגיל 10

שאלה 1: ברצוננו ליצור "תכונה" באמצעות המחלקה `Property<V>`, כאשר V יכול להיות כל טיפוס. נרצה שתהיה לנו האפשרות לכתוב אובייקטים מהסוג של `Property` כך כאשר האחד משנה את ערכו, כל האובייקטים הכרוכים אליו ישנו אוטומטית את ערכם בהתאמה. לדוגמא (ראו את ה main הבא):

ניצור את המשתנים `msp0`, `msp1`, `msp2` מסוג `Property<String>`. נכתוב את `msp1` ל `msp0`. נכתוב את `msp2` ל `msp1`. כשנשנה את ערכו של `msp0` ל "hello world!" אז גם `msp1` ישנה לאותו הערך, ומכיוון ש `msp2` כרוך ל `msp1` אז גם `msp2` ישנה בתורו את ערכו ל "hello world!" (טרנזיטיביות).

```
public static void main(String[] args) {  
    Property<String> msp0=new Property<String>();  
    Property<String> msp1=new Property<String>();  
    Property<String> msp2=new Property<String>();  
  
    msp0.bind(msp1); // when msp0 changes, so does msp1  
    msp1.bind(msp2);  
  
    msp0.setValue("hello world!");  
  
    System.out.println(msp2.getValue()); // hello world!  
}
```

ב package test בממשו את המחלקה Property<V> כך שה main לעיל יעבוד בהתאם לציפיות (שינוי msp0 גורר שינוי אוטומטי וטרמיטיבי ל msp2).

- בפרט, עליכם לממש את המתודות bind, setValue ו getValue.
- תשימו לב שע"פ הגדרת השאלה ניתן לכרוך לאובייקט Property כמה Properties שנרצה, לדוגמא נכרוך את msp1, msp2, msp3 ל msp0:
 - msp0.bind(msp1); msp0.bind(msp2); msp0.bind(msp3);
 - כש msp0 ישנה את ערכו לכל השאר יוזן ערך זה אוטומטית.
- הכריכה תהיה אך ורק ל Property עם אותו הטיפוס V.
 - למשל, אם היינו מנסים לכרוך משתנה מסוג Property<Integer> למשתנה מסוג Property<String> אז נרצה לקבל על כך שגיאת קומפילציה.

טיפ: חישבו על design pattern מתאים ומי ממלא איזה תפקיד

שאלה 2:

נתונה המחלקה Controller החושפת ממשק פקודה שכולל אחזור של ID לצורך זיהוי הפקודה.

```
package test;

public class Controller {

    public interface Command{
        int getID();
        void execute();
    }

    PriorityQueue<_____> queue;

    public Controller() {}

    public void insertCommand(Command c){}

    public void executeOne(){
        if(!queue.isEmpty())
            queue.poll().execute();
    }

    public void executeAll(){
        while(!queue.isEmpty())
            queue.poll().execute();
    }
}
```

מטרת ה Controller היא למזער את הזמן שהפקודות ממתינות בתור. לפיכך, יש להריץ את הפקודות ע"פ הזמן שלוקח להן לרוץ – מהזמן הקצר ביותר ועד לזמן הארוך ביותר. אולם, לא ניתן להעריך את הזמן שלוקח לפקודה לרוץ ללא מדידה.

לכן, נגדיר את המנגנון הבא. נשתמש בתור עדיפויות כדי להעדיף פקודות עם זמן ריצה קצר יותר. עבור פקודות שלא הרצנו בעבר (ID חדש) נניח שזמן הריצה שלהן הוא 0. בכל פעם שנריץ פקודה כלשהי נמדוד את זמן הריצה שלה. נתון זה ישמש אותנו לפעם הבאה שנתבקש להריץ את אותה הפקודה – כך כל פקודה תתמקם בתור לפי זמן הריצה שלה בפעם הקודמת. **לדוגמא** הכנסנו את הפקודות 1,2,3, וקראנו ל `executeAll`. הפקודות ירוצו (בסדר לא חשוב) וזמן הריצה של כל אחת יימדד. נניח ש 3 היא הקצרה ביותר, ו 1 היא הארוכה ביותר. לכן בפעם הבאה שנבקש להריץ את הפקודות סדר הריצה יהיה 3 ואז 2 ואז 1.

שימו לב שחל איסור לשנות את הממשק.

עליכם להשלים את הקוד לעיל כדי לממש מנגנון זה. בפרט, עליכם לממש את `insertCommand` עבור הכנסת פקודות ל `Controller` ע"פ המדיניות שהגדרנו.

טיפים:

- כדי למדוד זמן ריצה יש לדגום את שעון המחשב. בדקו מה עושה זאת במחלקה `System`.
- ע"פ הממשק, אובייקט מסוג `Command` לא נדרש למדוד לעצמו את זמן הריצה. ייתכן ותצטרכו ליצור מחלקת עזר פרטית (יתכנו מס' פתרונות שונים).
- אולי תצטרכו מבנה נתונים נוסף כדי לקשר בין פקודות לזמני ריצה (כנ"ל).

הגשה:

ל ex10 את `Property.java`, `Controller.java`

בהצלחה!